

LAB EXPERIMENT – 3

Water Jug Problem using Python — CO2

Aim

To write a Python program to solve the **Water Jug Problem** using a **Breadth-First Search (BFS)** approach.

Procedure

1. Consider two jugs with given capacities.
2. Define the initial state as (0, 0).
3. Define the required target amount of water.
4. Generate possible operations:
 - Fill the first jug.
 - Fill the second jug.
 - Empty the first jug.
 - Empty the second jug.
 - Pour water from the first jug into the second.
 - Pour water from the second jug into the first.
5. Use BFS to explore the possible states.
6. Store visited states to avoid repetition.
7. Continue until the target amount is obtained.
8. Display the sequence of operations.

Source Code

main.py

Visualize

Run ▶

Output

```
1 from collections import deque
2
3 def water_jug(capacity1, capacity2, target):
4     queue = deque()
5     visited = set()
6
7     # Initial state
8     queue.append(((0, 0), []))
9     visited.add((0, 0))
10
11 while queue:
12     (jug1, jug2), path = queue.popleft()
13
14     # Check goal
15     if jug1 == target or jug2 == target:
16         return path + [(jug1, jug2)]
17
18     states = [
19         ((capacity1, jug2), "Fill Jug 1"),
20         ((jug1, capacity2), "Fill Jug 2"),
21         ((0, jug2), "Empty Jug 1"),
22         ((jug1, 0), "Empty Jug 2")
23     ]
24
25     # Pour Jug 1 -> Jug 2
26     amount = min(jug1, capacity2 - jug2)
27     states.append(
28         ((jug1 - amount, jug2 + amount),
29          "Pour Jug 1 into Jug 2")
30     )
31
32     # Pour Jug 2 -> Jug 1
33     amount = min(jug2, capacity1 - jug1)
34     states.append(
35         ((jug1 + amount, jug2 - amount),
36          "Pour Jug 2 into Jug 1")
37     )
38
39     for state, action in states:
40         if state not in visited:
41             visited.add(state)
42             queue.append((state, path + [(action, state)]))
43
```

Water Jug Solution:

```
('Fill Jug 2', (0, 3))
('Pour Jug 2 into Jug 1', (3, 0))
('Fill Jug 2', (3, 3))
('Pour Jug 2 into Jug 1', (4, 2))
(4, 2)
```

main.py	Visualize	Run ▶	Output
<pre>17 18 states = [19 ((capacity1, jug2), "Fill Jug 1"), 20 ((jug1, capacity2), "Fill Jug 2"), 21 ((0, jug2), "Empty Jug 1"), 22 ((jug1, 0), "Empty Jug 2") 23] 24 25 # Pour Jug 1 -> Jug 2 26 amount = min(jug1, capacity2 - jug2) 27 states.append(28 ((jug1 - amount, jug2 + amount), 29 "Pour Jug 1 into Jug 2") 30) 31 32 # Pour Jug 2 -> Jug 1 33 amount = min(jug2, capacity1 - jug1) 34 states.append(35 ((jug1 + amount, jug2 - amount), 36 "Pour Jug 2 into Jug 1") 37) 38 39 for state, action in states: 40 if state not in visited: 41 visited.add(state) 42 queue.append((state, path + [(action, state)])) 43 44 return None 45 46 47 capacity1 = 4 48 capacity2 = 3 49 target = 2 50 51 solution = water_jug(capacity1, capacity2, target) 52 53 if solution: 54 print("Water Jug Solution:\n") 55 56 for step in solution: 57 print(step) 58 else: 59 print("No solution exists.")</pre>			Water Jug Solution: <pre>('Fill Jug 2', (0, 3)) ('Pour Jug 2 into Jug 1', (3, 0)) ('Fill Jug 2', (3, 3)) ('Pour Jug 2 into Jug 1', (4, 2)) (4, 2)</pre>

Result

Thus, the Water Jug Problem was successfully solved using Python and BFS.